Software Architecture

Design Pattern 1 – Singleton, Object Pool, Iterator

Practice Week 10

2025학년도 2학기

목요일 13:00 – 15:00

신공학관 1214호

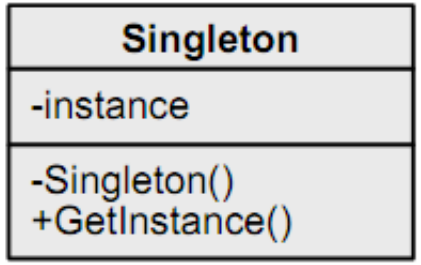
Konkuk Univ
DMS LAB

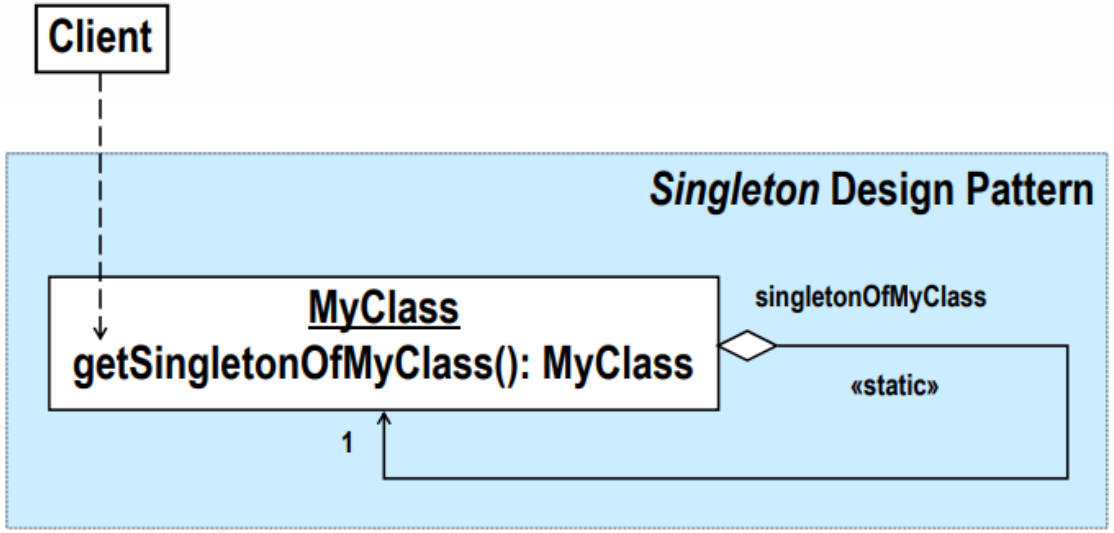
Design Pattern 1

- **Singleton**

Singleton (싱글턴)

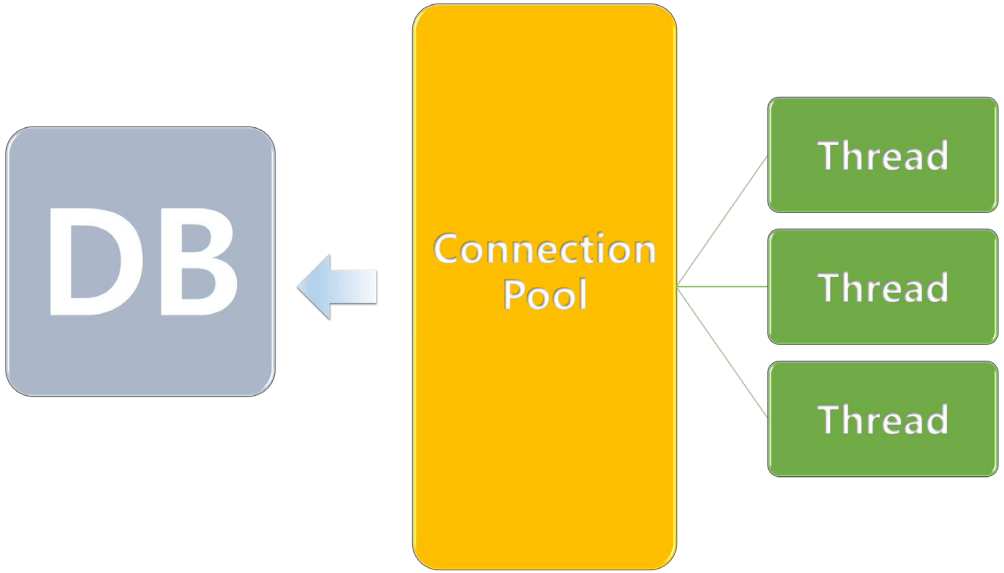
- **하나의 클래스에서 오직 하나의 인스턴스만 생성**되도록 보장하기 위하여, 클래스 외부에서 누구도 객체의 인스턴스를 생성하지 못하게 하는 패턴
- 클래스 인스턴스 하나만 만들고, 그 **인스턴스로의 전역접근을** 제공하는 패턴
- 하나의 클래스 당 오직 하나의 인스턴스 만을 가지고 작업하고 인스턴스를 제어하는 단 하나의 방법만을 제공하고자 할 때 유용하다.
  - 시스템에서 오직 하나의 윈도우 관리자를 가지거나 혹은 하나의 프린터 스풀러, 데이터 베이스를 제어하는 하나의 리소스를 가지는 경우에 유용하다.





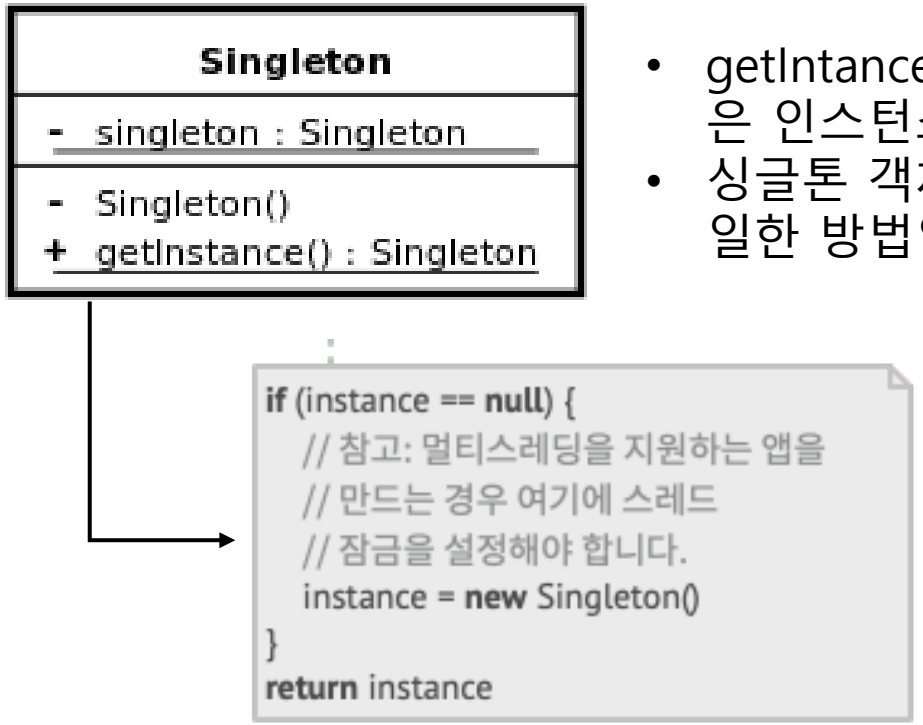
Singleton (싱글턴)

- 예시) DB Connection Pool (Thread Pool)
  - 하나의 DB에 대하여 여러 개의 Connection객체를 생성하지 않고, 하나의 객체에 여러 개의 Thread를 연결시킴
  - Singleton Pattern 적용 → 객체가 1개만 생성되도록 보장
  - Object Pool Pattern 적용 → Thread를 미리 생성, 저장하여 필요에 따라 처리 Task의 수를 조절



Singleton (싱글턴)

- 기본 구조



- getIntance()는 자체 클래스의 같은 인스턴스를 반환함
- 싱글톤 객체를 가져올수 있는 유일한 방법임

Singleton (싱글턴)

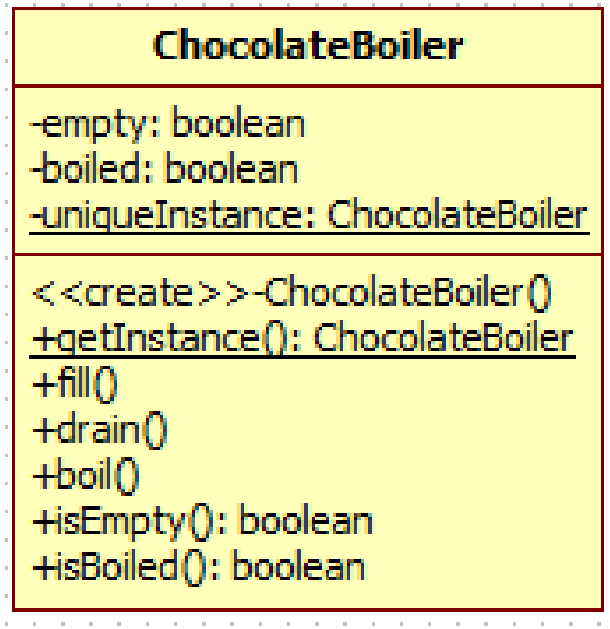
- 사용법
  - 객체를 딱 하나만 생성하여 생성된 객체를 프로그램 어디에서나 접근하여 사용할 수 있도록 하는 패턴
  - 리소스를 많이 차지하는 무거운 객체를 한번만 선언하고 사용

- 장점
  - **메모리 낭비를 방지**할 수 있음
  - 싱글턴으로 만들어진 클래스와 다른 클래스의 인스턴스들의 데이터 공유가 쉬움
  - 인스턴스가 절대적으로 한 개만 존재하는 것을 보증하기에 개발 시 실수를 줄일 수 있음
  - 싱글턴 객체를 사용하지 않는 경우 인스턴스를 생성하지 않음
  - 싱글턴을 상속시킬 수 있음

- 단점
  - 전역변수보다 사용하기가 불편함
  - 싱글턴의 역할이 커질수록 결합도가 높아져 **객체 지향 설계 원칙에 어긋날 수 있음**
  - **멀티쓰레드 환경에서 컨트롤이 어려울 수 있음**

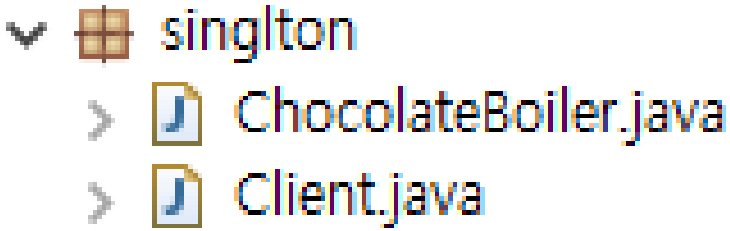
실습: Singleton

- Chocolate Boiler: 초콜릿을 끓이는 장치
  - Fill(): empty = true 이면 재료를 넣는다.
  - Drain(): empty = false, boiled = true 이면 boiler를 비운다.
  - Boil(): empty = false, boiled = false 이면 끓인다.



실습: Singleton

- **Create JAVA Project with packages**
  - Put the generated codes into “singleton” package



실습: Singleton

- Client.java

```java
package singlton;

public class Client {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        ChocolateBoiler boiler1 = ChocolateBoiler.getInstance();
        ChocolateBoiler boiler2 = ChocolateBoiler.getInstance();
        ChocolateBoiler boiler3 = ChocolateBoiler.getInstance();

        System.out.println("=====Check Singlton=====");
        System.out.println(boiler1.toString());
        System.out.println(boiler2.toString());
        System.out.println(boiler3.toString());

        System.out.println("\n=====Chocolate Boilor=====");
        boiler1.fill();
        boiler1.boil();
        boiler1.drain();
    }

}
```

new ChocolateBoiler() (생성자)를 호출하지 않고, getInstance()라는 특수한 메서드를 통해 객체를 얻어옴. 3번을 호출했지만, 우리는 이 3개가 모두 같은 객체이길 기대함

boiler1, boiler2, boiler3의 메모리 주소를 출력함. 만약 싱글톤 패턴이 올바르게 작동했다면, 이 3줄은 완전히 동일한 주소값을 출력함

boiler1을 조작하면, boiler2와 boiler3도 같은 객체이므로 동일한 상태를 공유하게 됩니다

실습: Singleton

- ChocolateBoiler.java

```java
package singlton;

public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;

    private static ChocolateBoiler uniqueInstance;

    private ChocolateBoiler() {
        empty = true;
        boiled = false;
    }

    public static synchronized ChocolateBoiler getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new ChocolateBoiler();
        }
        return uniqueInstance;
    }
```

static 을 이용하여 이 uniqueInstance 변수를 '클래스' 레벨에 단 하나만 존재함.

생성자를 private으로 선언. 외부 클래스에서 new ChocolateBoiler호출을 차단함

** Synchronized: 한 스레드가 메소드 사용을 끝내기 전까지 다른 스레드는 기다림 → 멀티스레딩 문제 해결

초콜릿보일러를 작동하는 비즈니스 로직(기능) 코드

```java
    public void fill() {
        if (isEmpty()) {
            System.out.println("Fill ChocolateBoiler");
            empty = false;
            boiled = false;
            System.out.println("-Empty:" + empty + ", boiled:" + boiled);
        }
    }

    public void drain() {
        if (!isEmpty() && isBoiled()) {
            System.out.println("Drain ChocolateBoiler");
            empty = true;
            System.out.println("-Empty:" + empty + ", boiled:" + boiled);
        }
    }

    public void boil() {
        if (!isEmpty() && !isBoiled()) {
            System.out.println("Boil ChocolateBoiler");
            boiled = true;
            System.out.println("-Empty:" + empty + ", boiled:" + boiled);
        }
    }

    public boolean isEmpty() {
        return empty;
    }

    public boolean isBoiled() {
        return boiled;
    }
}
```

실습: Singleton

- Result

```
=====Check Singlton=====
singlton.ChocolateBoiler@5305068a
singlton.ChocolateBoiler@5305068a
singlton.ChocolateBoiler@5305068a

=====Chocolate Boilor=====
Fill ChocolateBoiler
-Empty:false, boiled:false
Boil ChocolateBoiler
-Empty:false, boiled:true
Drain ChocolateBoiler
-Empty:true, boiled:true
```

동일한 인스턴스

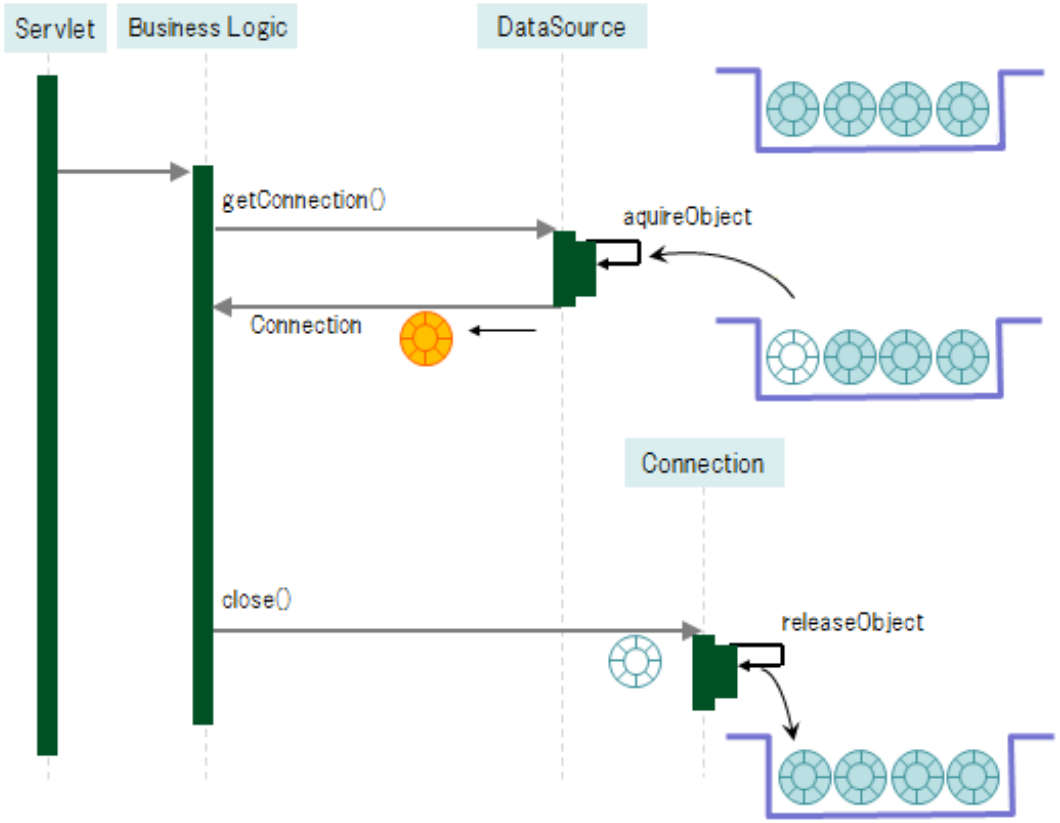
Design Pattern 1

- **Object Pool**

디자인 패턴 : Object Pool

Object Pool

- 생성할 객체가 너무 시간이 오래 걸리거나 생성할 수 있는 객체 수가 제한된 경우에 객체를 컬렉션 객체(ArrayList, Vector...)에 넣어놓고 재사용 할 수 있도록 관리한다.
  - **대표적인 예 :** Thread pool, DB Connection pool

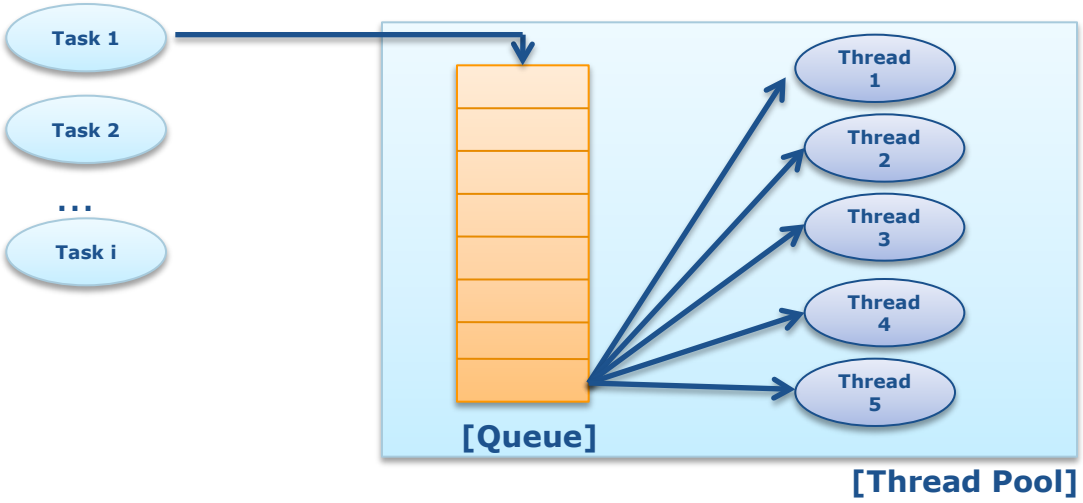


Object Pool Pattern

- 사용법
  - 객체를 빈번하게 생성/삭제 해야 할 때
  - 객체들의 크기가 비슷할 때
  - 객체를 힙에 생성하기엔 느리거나 메모리 단편화가 우려될 때
  - DB 연결이나 네트워크 연결같이 객체 생성시 비용이 비쌀 경우 미리 생성해두고 재사용할 필요가 있을 때
  - 객체 풀 클래스에 들어가는 객체는 현재 자신이 사용 중인지 아닌지 알 수 있는 방법을 제공해야함
    - Pool은 초기화될 때 사용할 객체들을 미리 생성하여 배열에 넣어두고 사용 안함 상태로 초기화
    - 새로운 객체가 필요하면 Pool에 요청해야 함
    - Pool은 사용 가능한 객체를 찾아 사용 중으로 초기화 한 뒤 리턴
    - 객체를 더 이상 사용하지 않는다면 사용 안 함 상태로 되돌려야 함
- 장점
  - 메모리를 신경쓰지 않고 마음껏 객체를 생성, 삭제할 수 있음
- 단점
  - 객체 아무것도 사용하지 않아도 메모리를 차지하고 있는 상태가 됨

Object Pool Pattern
- Thread Pool Example

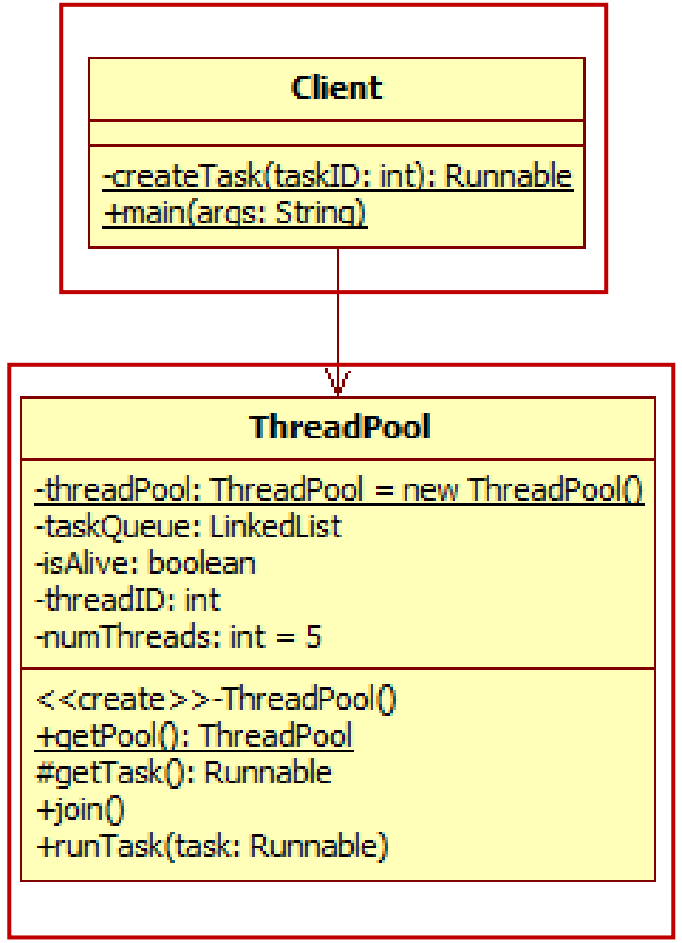
- Object Pool
  - 실습 (Thread Pool)
    - Singleton pattern이 추가된 실습
    - Thread Pool은 미리 Thread를 생성 및 저장하여 필요에 따라 작업을 처리하기 위한 Pool
    - Thread Pool의 구현은 용도에 따라 다양함
      - 소켓 통신에서 많이 쓰임



Object Pool Pattern

- Thread Pool Example

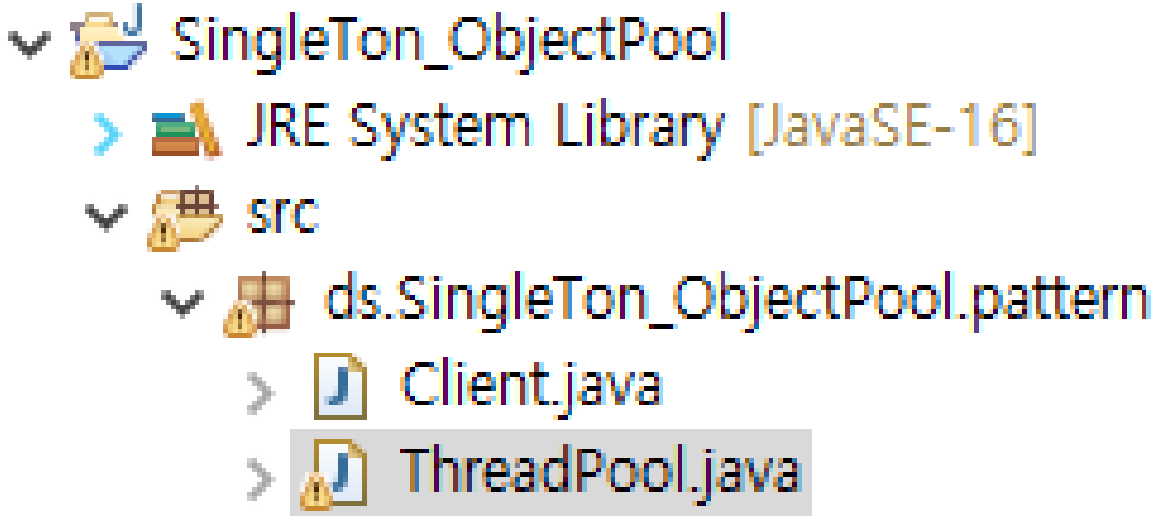
ThreadPool 객체를 사용하는 역할



제한된 시간 동안 다른 객체와 협업하는 역할

Object Pool Pattern
- Thread Pool Example

- **Create JAVA Project with packages**
  - Put the generated codes into “ds.SingleTon_ObjectPool.pattern” package



Object Pool Pattern

- Thread Pool Example : Client.java

```java
package ds.SingleTon_ObjectPool.pattern;

public class Client {
    private static Runnable createTask(final int taskID) {
        return new Runnable() {
            public void run() {
                System.out.println("Task " + taskID + ": start");

                try {
                    Thread.sleep(500);

                } catch (InterruptedException ex) {  }

                System.out.println("Task " + taskID + ": end");
            }
        };
    }

    public static void main(String args[]) {
        int numTasks = 8;

        ThreadPool threadPool = ThreadPool.getPool();

        for(int i = 0; i < numTasks; i++)
            threadPool.runTask(createTask(i));

        threadPool.join();
    }
}
```

각 작업은 0.5초간 지속

총 8개의 Task를 생성 하여 작업 시작

8개의 일이 끝날때까지 대기하는 비즈니스 로직

Object Pool Pattern
- Thread Pool Example : ThreadPool.java(1/3)

Singleton Pattern!
(static을 이용하여 threadpool은 1개만)

```java
package ds.SingleTon_ObjectPool.pattern;

import java.util.LinkedList;

public class ThreadPool extends ThreadGroup{

    private static ThreadPool threadPool = new ThreadPool();

    private LinkedList taskQueue;
    private boolean isAlive;
    private int threadID;

    private int numThreads = 5;

    private ThreadPool() {
        super("ThreadPool");

        isAlive = true;
        taskQueue = new LinkedList();

        for (int i = 0; i < numThreads; i++)
            new PooledThread().start();
    }

    public static ThreadPool getPool() {
        return threadPool;
    }
```

Pool의 크기는 '5'이다

작업을 수행할 수 있는 Thread Pool을 생성한다

Singleton Pattern 으로 생성된 객체를 가져올때 사용

Object Pool Pattern
- Thread Pool Example : ThreadPool.java(2/3)

```
29    protected synchronized Runnable getTask() throws InterruptedException {
30
31        while(taskQueue.size() == 0) {
32            if(!isAlive) {
33                return null;
34            }
35            wait();
36        }
37        return (Runnable)taskQueue.removeFirst();
38    }
39
40    public void join() {
41
42        synchronized(this) {
43            isAlive = false;
44            notifyAll();
45        }
46
47        Thread[] threads = new Thread[activeCount()];
48
49        int count = enumerate(threads);
50        for(int i = 0; i < count; i++) {
51            try {
52
53                threads[i].join();
54
55            } catch (InterruptedException ex) {  }
56        }
57    }
58
```

첫 Task가 수행되어 끝나면 기다렸다가
큐(Pool)의 첫 작업을 빼냄

Task 끝날 때 호출

현재 수행되고 있는 Task 수

Task가 끝날 때 까지 기다림

Object Pool Pattern
- Thread Pool Example : ThreadPool.java(3/3)

```java
    public synchronized void runTask(Runnable task) {
        if(task != null) {

            taskQueue.add(task);
            notify();
        }
    }

    private class PooledThread extends Thread {
        public PooledThread() {
            super(ThreadPool.this, "PooledThread-" + (threadID++));
        }

        public void run() {
            while(!isInterrupted()) {
                Runnable task;
                try {

                    task =getTask();

                    if(task == null)
                        return;
                    else

                        task.run();

                } catch (InterruptedException ex) {
                    ex.toString();
                }
            }
        }
    }
}
```

Task를 큐(Pool)에 넣음

큐(Pool)에서 작업할 Task를 가져와 처리

Object Pool Pattern
- Thread Pool Example: Result

```
Task 3: start
Task 0: start
Task 4: start
Task 1: start
Task 2: start
Task 3: end
Task 5: start
Task 1: end
Task 2: end
Task 4: end
Task 0: end
Task 7: start
Task 6: start
Task 5: end
Task 6: end
Task 7: end
```

Design Pattern 1

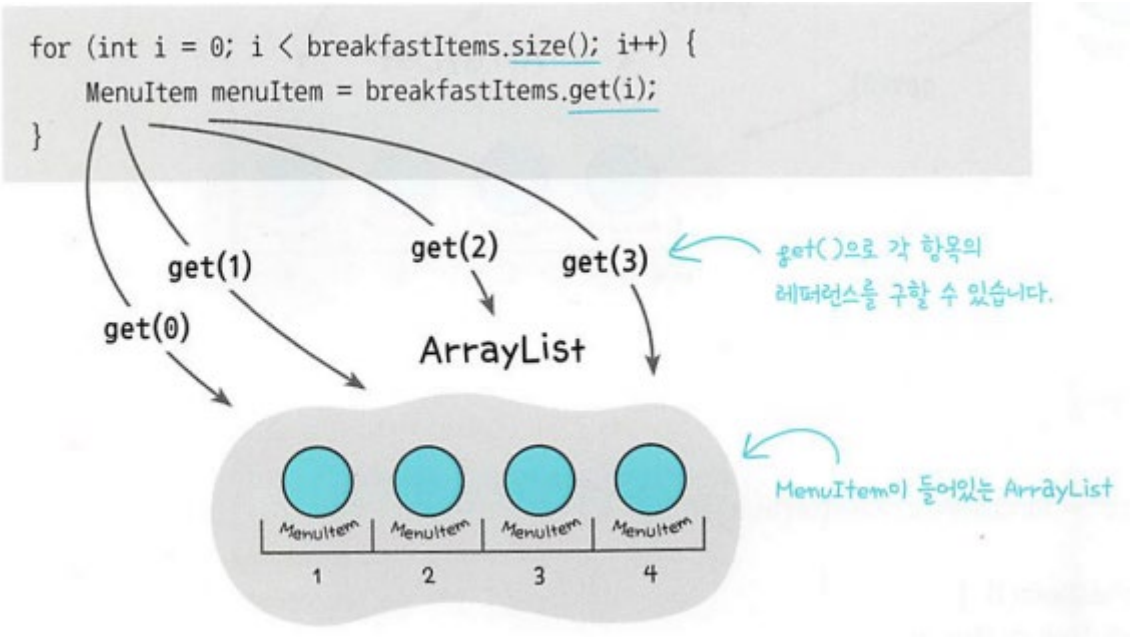
- **Iterator**

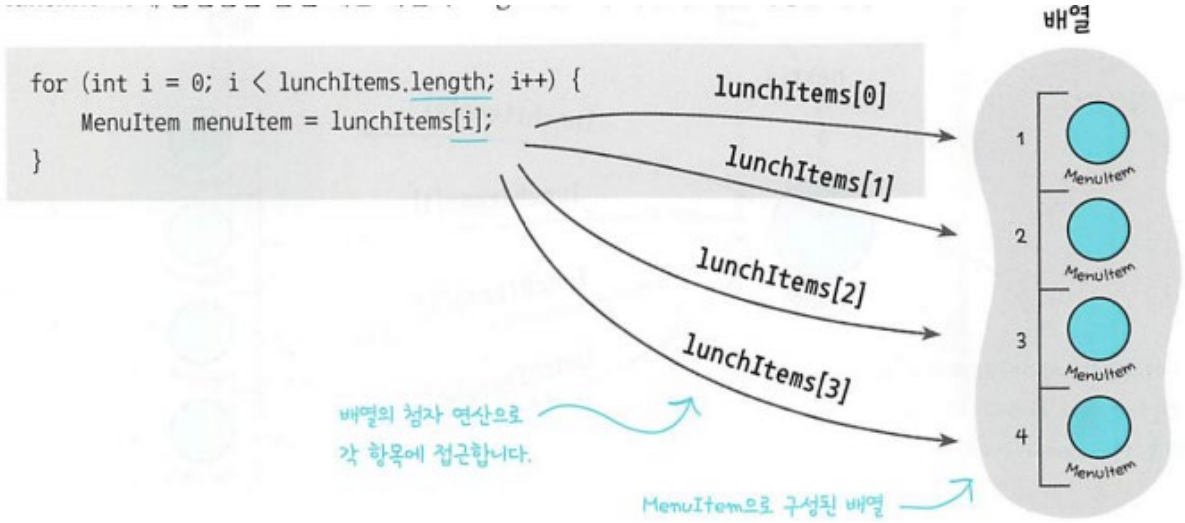
디자인 패턴 : Iterator

Iterator (=반복, 열거) – 어떻게 감추고, 무엇을 드러낸다.

- 배열과 같은 집합체에서 하나씩 끄집어 내어 열거 하면서 전체를 검색하는 처리를 말함
- 예시) 책장의 책을 보려는데, 책장의 구조를 알아야함
  - 둘 다 for문을 이용해 반복 작업을 할 수 있지만,
  - ArrayList는 size(), get() 메소드 사용 / Array라면 array[index]를 사용

Iterator 패턴 적용 전





Iterator (반복자)

- 예시) ArrayList와 Array
  - 둘 다 for문을 이용해 반복 작업을 할 수 있다.
  - **Iterator로 캡슐화**

Iterator 패턴 적용 후

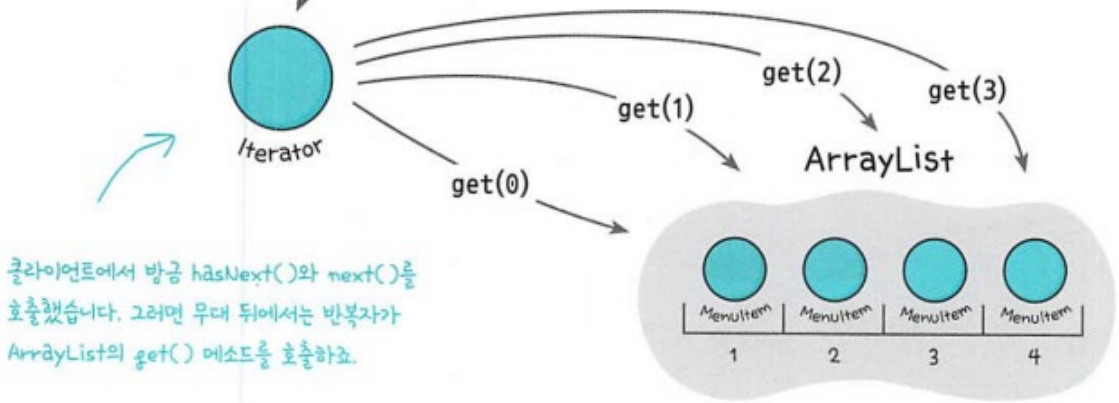
```
Iterator iterator = breakfastMenu.createIterator();

while (iterator.hasNext()) {
    MenuItem menuItem = iterator.next();
}
```

breakfastMenu에 들어있는 MenuItem에 반복자(Iterator 객체)를 요구합니다.

아직 항목이 남아있는 동안...

다음 항목을 뽑아냅니다.

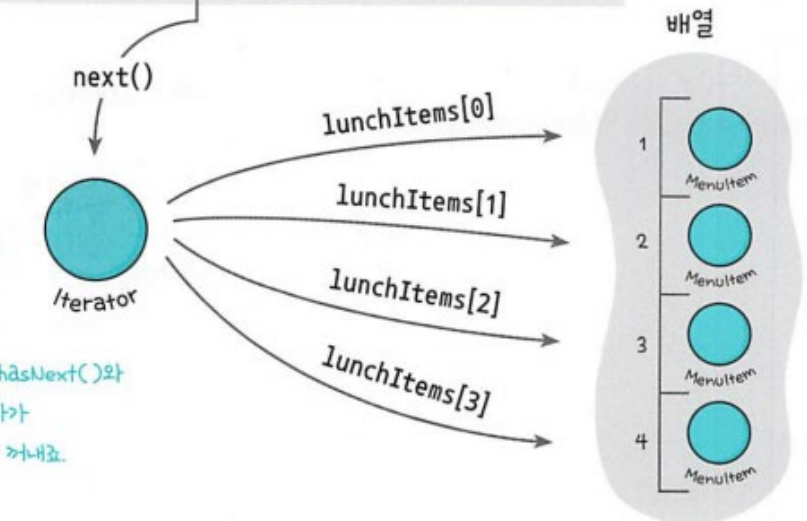


클라이언트에서 방금 hasNext()와 next()를 호출했습니다. 그러면 무대 뒤에서는 반복자가 ArrayList의 get() 메소드를 호출하죠.

```
Iterator iterator = lunchMenu.createIterator();

while (iterator.hasNext()) {
    MenuItem menuItem = iterator.next();
}
```

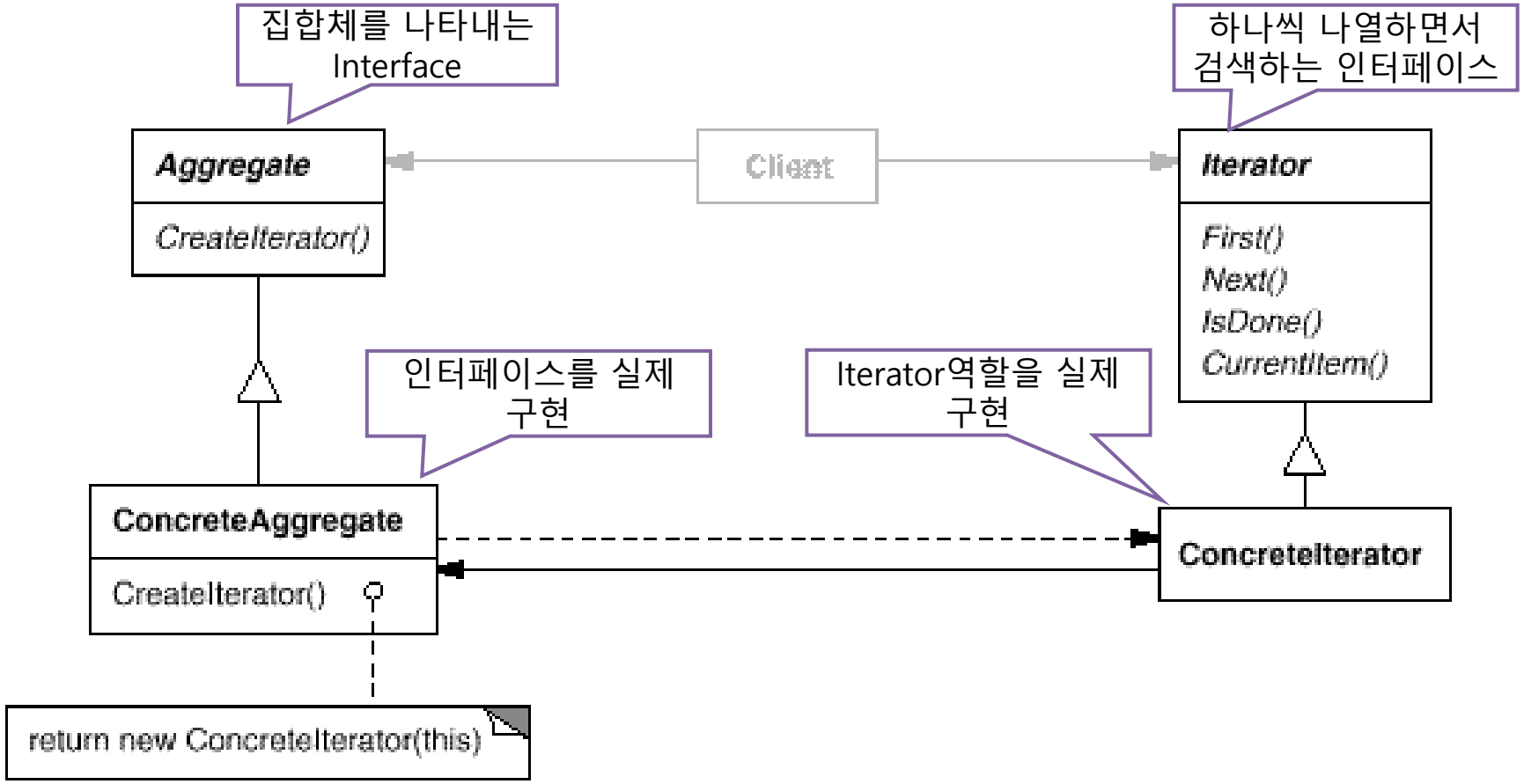
이 코드는 breakfastMenu 코드와 완전히 같습니다.



여기서도 마찬가지입니다. 클라이언트에서 hasNext()와 next()를 호출하면, 무대 뒤에서는 반복자가 배열의 인덱스를 써서 MenuItem 항목을 꺼내죠.

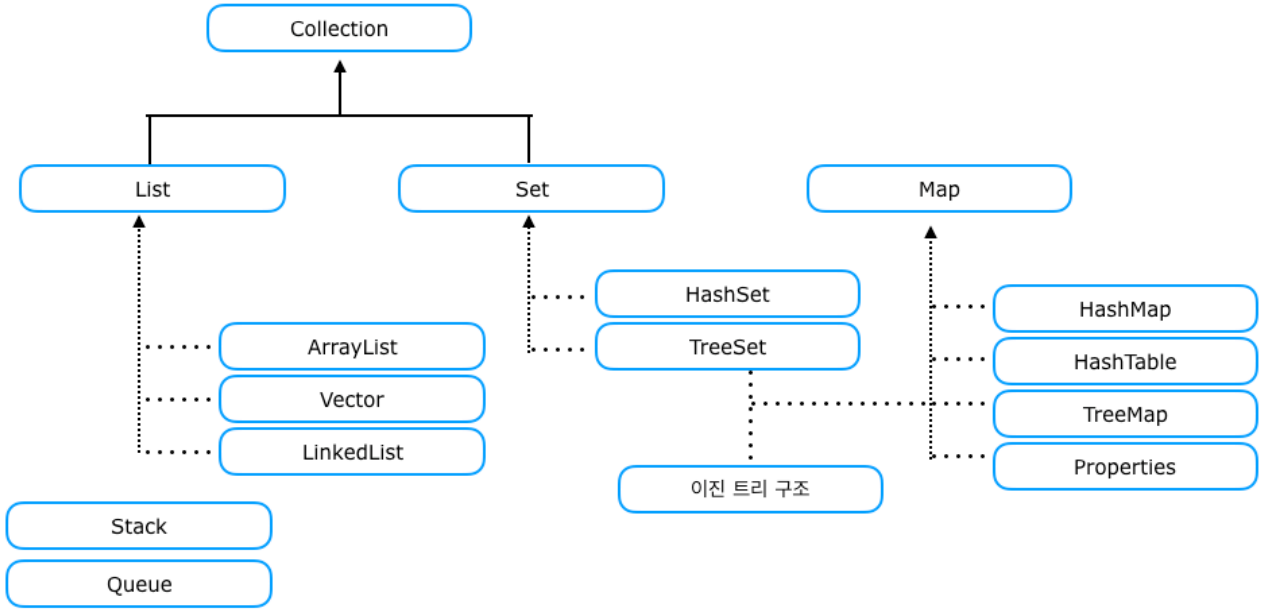
Iterator Pattern

- 기본 클래스 다이어그램



Java Collection Framework

- 배열은 생성할 때, 크기가 고정되며, 그 크기가 넘어가면 더 이상 데이터를 저장할 수 없음. 또한, 해당 index의 데이터를 삭제하면, 메모리가 낭비 될 수 있음
- 따라서 Java에서는 이러한 배열의 문제를 해결하기위해, 널리 알려진 자료구조(List, Set, Map, Stack, Queue등)들을 효율적으로 관리(추가, 삭제, 검색, 저장)할 수 있도록 Collection Framework 지원하고 있음

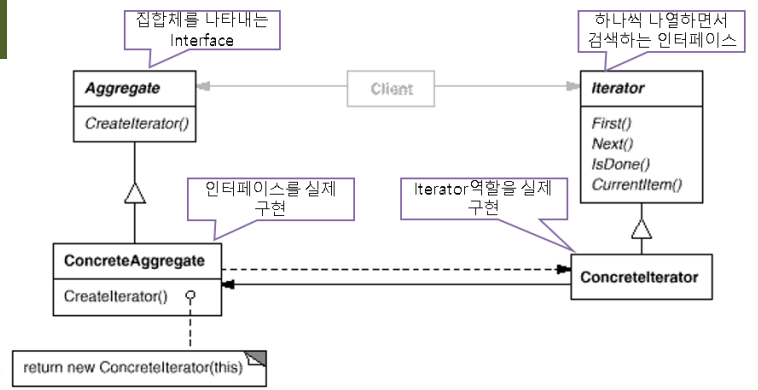


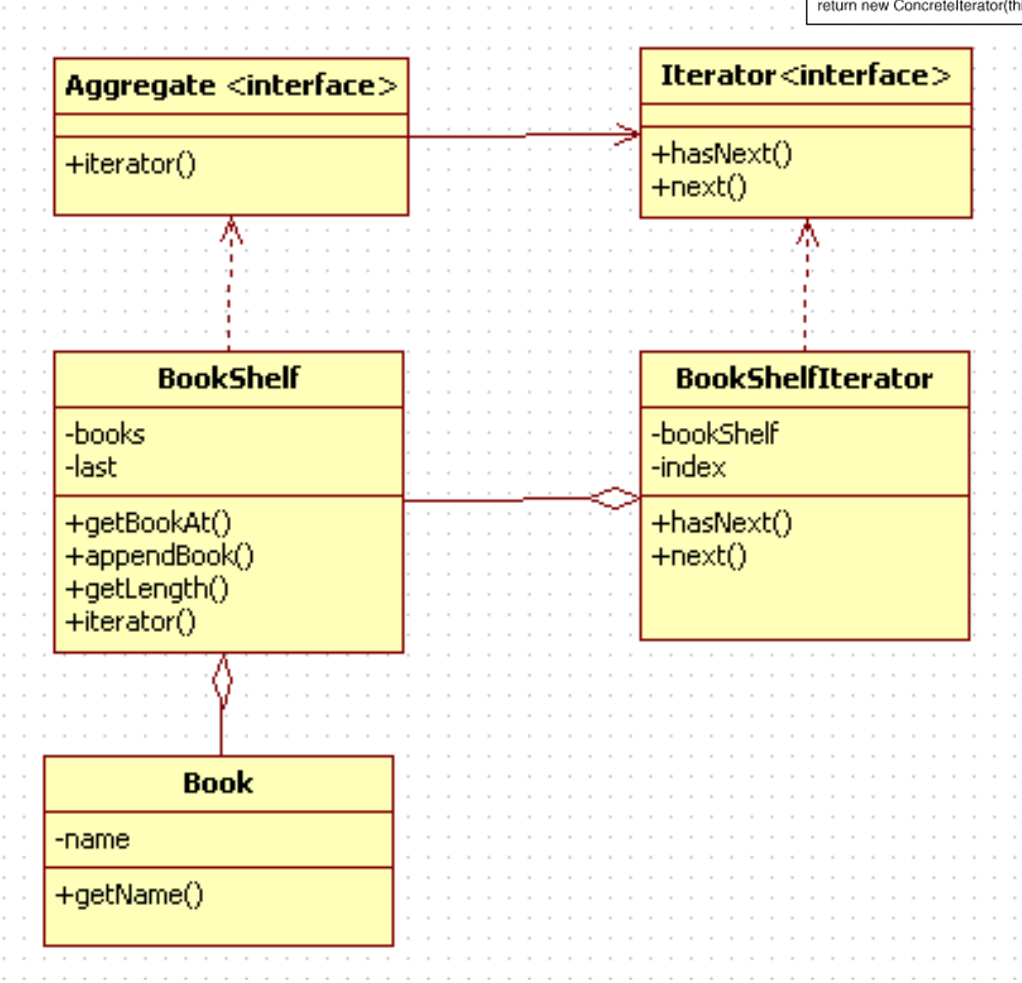
Iterator Pattern

- 사용법
  - Collection이나 집합 객체의 내부적인 구현 방법을 드러내지 않으면서도 그 안에 있는 모든 항목(데이터)에 순차적으로 접근
- 장점
  - 구조가 복잡한 집합 객체 내에서 어떻게 일이 처리되는지 몰라도 안에 들어 있는 모든 항목(데이터)에 대해 반복 작업 수행 가능
  - Collection 객체 내에 모든 항목 (데이터) 에 접근하는 방식이 통일되어 다형성을 높여주는 코드가 될수 있음
  - 모든 항목(데이터)에 접근하는 작업을 Collection 객체가 아닌 Iterator 객체에서 맡게 되므로 구현이 간단해짐
- 단점
  - 단순한 순회를 구현하는 경우 클래스만 많아져 복잡도가 증가할 수 있음
  - 대량의 데이터를 제어할 때 속도가 느릴 수 있음

실습: Iterator

- BookShelf: 책장에 있는 책들의 제목을 읽기



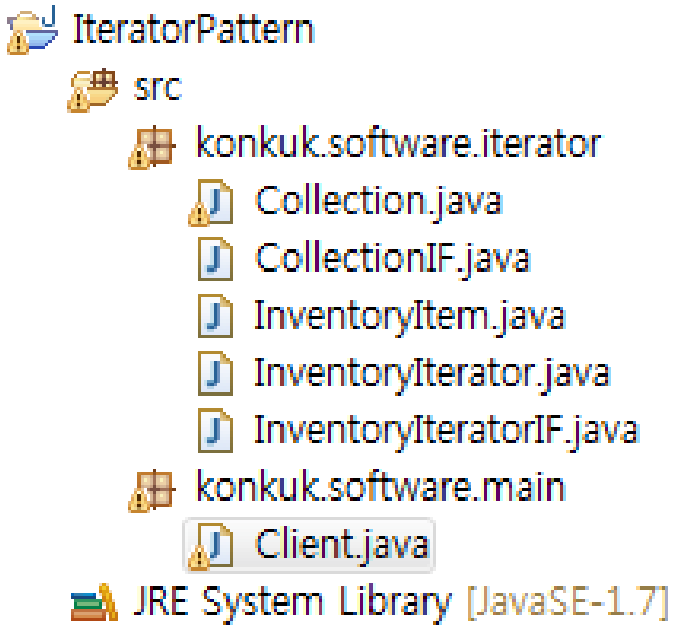


hasNext(): 대상이 더 있는지 확인

next(): 다음 객체를 리턴

Iterator Pattern
- BookShelf Example

- **Create JAVA Project with packages**
  - Put the generated codes into “konkuk.software.iterator” package



Iterator Pattern
- BookShelf Example: Client.java

```java
public class Client {

    /**
     * @param args
     */
    // ## operation main(String[])
    public static void main(String[] args) {
        // #[ operation main(String[])
        // #]
        Collection bs = new Collection(4);
        bs.setBookAt(new InventoryItem("노인과바다"));
        bs.setBookAt(new InventoryItem("논리야놀자"));
        bs.setBookAt(new InventoryItem("수학의정석"));
        bs.setBookAt(new InventoryItem("Patterns in Java Volume 1"));

        InventoryIteratorIF it = bs.iterator();

        while (it.hasNext()) {
            System.out.println(((InventoryItem) it.next()).getName());
        }

}
```

4개의 책을 꽂기

Iterator를 이용해 제목을 읽어보자
bs(책장)에게 Iterator()를 호출함

Iterator Pattern
- BookShelf Example: CollectionIF, InventoryIteratorIF

책장 역할(사서) Interface ▼

```
public interface CollectionIF {


    //## operation interator()
    InventoryIteratorIF iterator();

}
```

책장 도우미(북카트) Interface ▼

```
public interface InventoryIteratorIF {


    //## operation hasNext()
    boolean hasNext();

    //## operation next()
    Object next();

}
```

Iterator Pattern
- BookShelf Example: Collection.java

책장 구현체 ▼

실제 책장을 배열로 만듦

```java
public class Collection implements CollectionIF {

    private InventoryItem[] books;      //## attribute books

    private int last = 0;       //## attribute last


    // Constructors

    /**
     * @param maxsize
     */
    //## operation Collection(int)
    public  Collection(int maxsize) {
        //#[ operation Collection(int)
        //#]
        this.books = new InventoryItem[maxsize];
    }

    /**
     * @param inx
     */
    //## operation getBookAt(int)
    public InventoryItem getBookAt(int inx) {
        //#[ operation getBookAt(int)
        //#]
        return books[inx];
    }
```

특정 장소의 책을 가져옴

책의 개수

```java
    //## operation getLength()
    public int getLength() {
        //#[ operation getLength()
        //#]
        return last;
    }
```

특정 장소에 책 꽂기

```java
    /**
     * @param book
     */
    //## operation setBookAt(InventoryItem)
    public void setBookAt(InventoryItem book) {
        //#[ operation setBookAt(InventoryItem)
        //#]
        this.books[last] = book;
        last++;
    }
```

책들 설정

```java
    /**
     * @param books
     */
    //## operation setBooks(InventoryItem[])
    public void setBooks(InventoryItem[] books) {
        //#[ operation setBooks(InventoryItem[])
        //#]
        this.books = books;
    }
```

책은 iterator를 통해 읽어 들일 수 있다

```java
    @Override
    public InventoryIteratorIF iterator() {
        // TODO Auto-generated method stub
        return new InventoryIterator(this);
    }

}
```

Client가 iterator()를 호출하면, 책장은 "자신(this)의 책을 넘겨줄 책장 도우미"(InventoryIterator)를 ' 새로 생성해서 리턴해줌

Iterator Pattern
- BookShelf Example: InventoryItem.java

```java
public class InventoryItem {

    protected String name; // ## attribute name

    // Constructors

    /**
     * @param name
     */
    // ## operation InventoryItem(String)
    public InventoryItem(String name) {
        // #[ operation InventoryItem(String)
        // #]
        super();
        this.name = name;
    }

    // ## operation getName()
    public String getName() {
        // #[ operation getName()
        // #]
        return name;
    }
    public void setName(String name) {
        // #[ operation setName(String)
        // #]
        this.name = name;
    }

}
```

책 이름 가져오는 함수

Iterator Pattern
- BookShelf Example: InventoryIterator.java

책장 도우미의 담당할 책장과 기억할 index를 가지고 있음

```java
public class InventoryIterator implements InventoryIteratorIF {

    private Collection bookShelf; // ## attribute bookShelf

    private int index; // ## attribute index

    // Constructors

    /**
     * @param bookShelf
     */
    // ## operation InventoryIterator(Collection)
    public InventoryIterator(Collection bookShelf) {
        // #[ operation InventoryIterator(Collection)
        // #]
        this.bookShelf = bookShelf;
        this.index = 0;
    }

    // ## operation hasNext()
    public boolean hasNext() {
        // #[ operation hasNext()
        // #]
        if (index < bookShelf.getLength()) {
            return true;
        } else {
            return false;
        }
    }
```

책이 있는지 여부 반환

index보다 책장의 총 개수가 작을 경우 책이 더 남았다 라는 코드임

책 객체를 순차적으로 반환함

```java
    public Object next() {
        // #[ operation next()
        // #]
        InventoryItem book = bookShelf.getBookAt(index);
        index++;
        return book;
    }

}
```

Iterator Pattern
- BookShelf Example: Result

Problems | Javadoc | Declaration | Console

```
<terminated> Client (16) [Java Application] C:₩Program Fil
노인과바다
논리야놀자
수학의정석
Patterns in Java Volume 1
```

실습 제출

- 아래 실습 후 모든 파일을 압축
  - Singleton, Object Pool, Iterator

- 위 실습은 똑같이 따라 할 필요 없음
  - 클래스명이나 함수명등을 임의로 바꿔도 무관함
  - 클래스 다이어그램하고 PPT내의 Source Code가 상이할수 있음

- 제출할 것(학번-이름-Practice_10주차.zip)
  - 아래 파일들을 압축해서 업로드(실습 때 했던 것을 모두 압축하기)
    - 예)
      - Diagram 스크린샷
      - UML 파일
      - 소스코드

Thank you!

Any Questions?